

# REST

## *Representational State Transfer*

dr. Matej Šprogar

# Problem

Sodelovanje spletnih strani

v okviru iste aplikacije

med več aplikacijami

iz iste domene

iz različnih domen

Tehnologije/Arhitekture/Jeziki/Filozofija:

RPC, SOAP(WSDL, WS-\*), custom API, REST

# Bistvo

REST je ideja, *kako* naj se aplikacije **pogovarjajo**. REST ne govori o tem, kako naj aplikacija deluje oziroma kako naj bo zasnovana navznoter, ampak govori o tem, kakšna naj bo zasnova navzven, da bo enostavno uporabna komurkoli.

Aplikacija tipa odjemalec-strežnik temelji na sporočilih; ta so lahko razumljiva ali samo aplikacijski kodi ali tudi drugim programom. REST prenaša informacije o možnih ukazih.

Nič nas torej ne sili v REST, je pa z njim lažje našo aplikacijo vpeti v druge aplikacije/storitve, ne več zgolj v brskalnik.

# Na kratko...

1. Pomenska uporaba HTTP metod in HTTP statusnih kod.
2. *Stateless* strežniki in informirani odjemalci
  - a. Informirane povezave na možne akcije
  - b. Neodvisne zahteve z vso potrebno vsebino
3. URI-ji kot struktura map
  - a. Intuitivna drevesna oblika naslavljanja virov
4. Prenos podatkov z XML ali JSON ali obema

# Pomenska uporaba HTTP metod

| Metoda | Pomen                  | Primer                 |
|--------|------------------------|------------------------|
| GET    | Preberi vir            | GET /uporabniki/123    |
| POST   | Ustvari nov vir        | POST /uporabniki       |
| PUT    | Popolnoma posodobi vir | PUT /uporabniki/123    |
| PATCH  | Delno posodobi vir     | PATCH /uporabniki/123  |
| DELETE | Izbriši vir            | DELETE /uporabniki/123 |

# The 6 Guiding Principles of REST

<https://restfulapi.net/>

## 1. **Odjemalec–strežnik:**

Uporabniški vmesnik in shranjevanje podatkov sta ločena, kar izboljša prenosljivost in skalabilnost.

## 2. **Brez stanja (stateless):**

Vsaka zahteva mora vsebovati vse potrebne podatke – strežnik ne shranjuje podatkov o prejšnjih zahtevah.

## 3. **Predpomnjenje (cacheable):**

Odzivi morajo biti označeni kot predpomnjeni ali ne – za boljšo učinkovitost.

## 4. **Enoten vmesnik (uniform interface):**

Standardiziran način dostopanja do virov (GET, POST, PUT, DELETE, ...).

## 5. **Plastna arhitektura:**

Komponente so organizirane v plasteh – posamezna komponenta vidi le neposredno sosednjo plast.

## 6. **Koda na zahtevo (opcijsko):**

Strežnik lahko odjemalcu pošlje skripte ali aplete za razširitev funkcionalnosti.

# **HATEOAS: Hypermedia As The Engine Of Application State**

Klient dostopa do spletne aplikacije preko informacij v hipertekstu in ne potrebuje dodatnih specifičnih znanj, kot je to primer pri drugih tipih spletnih aplikacij/storitev.

Uporaba HTTP ukazov in zapisa stanj in akcij v hipertextu omogoča *rahlo sklopljenost* med klientom in strežnikom ter posledično lažji razvoj novih funkcionalnosti na strežniku, ki ne rušijo klientov.

*Vse možne akcije nekega stanja so vedno zapisane v trenutnem dokumentu, prehod med stanji poteka s pomočjo povezav.*

# Primer

```
GET /accounts/12345 HTTP/1.1
```

```
Host: bank.example.com
```

```
Accept: application/xml
```

```
***** odgovor *****
```

```
HTTP/1.1 200 OK
```

```
Content-Type: application/xml
```

```
Content-Length: ...
```

```
<?xml version="1.0"?>
```

```
<account>
```

```
  <account_number>12345</account_number>
```

```
  <balance currency="usd">100.00</balance>
```

```
  <link rel="deposit" href="https://bank.example.com/accounts/12345/deposit" />
```

```
  <link rel="withdraw" href="https://bank.example.com/accounts/12345/withdraw" />
```

```
  <link rel="transfer" href="https://bank.example.com/accounts/12345/transfer" />
```

```
  <link rel="close" href="https://bank.example.com/accounts/12345/close" />
```

```
</account>
```

```
***** ali *****
```

```
HTTP/1.1 200 OK
```

```
Content-Type: application/xml
```

```
Content-Length: ...
```

```
<?xml version="1.0"?>
```

```
<account>
```

```
  <account_number>12345</account_number>
```

```
  <balance currency="usd">-25.00</balance>
```

```
  <link rel="deposit" href="https://bank.example.com/accounts/12345/deposit" />
```

```
</account>
```



# Povzetek

Čeprav je REST osnovan na stateless protokolu, ima v imenu *prenos stanja* - govora je o aplikacijskem stanju, ki določa možne akcije v programu, in ki se v celoti prenese na klienta v preprosti obliki povezav. Vsaka zahteva je polno informirana!

REST je filozofija o tem, kako naj bodo strežniški viri predstavljeni na klientu.

Richardson Maturity Model

**Martin Fowler:**

<https://martinfowler.com/articles/richardsonMaturityModel.html>